Lecture 9 — Strings for Regionals

> Regional Level · Prerequisites: Year Curriculum Weeks 27 (interval DP), 28–29 (hashing, prefix function, Z-function, tries, Aho–Corasick) · Estimated time: 2.5 hours

1. Overview

String problems at regionals rarely require exotic structures. Most are solved by one standard tool plus one observation:

- "compare substrings quickly" → **polynomial hashing**, often combined with binary search;
- "palindromes" → **Manacher's algorithm**, or hashing in both directions;
- "split the string into pieces with a property" → **DP over prefixes**;
- "count strings of length n that avoid or contain a pattern" → **DP over the KMP automaton**.

This lecture covers each tool and the regional-style observations that go with it. Suffix arrays with LCP and suffix automata are in the National track (Lecture `nationals-06`); the palindromic tree is in the World Finals track.

2. Hashing done safely

2.1 The formula

With `h[i]` the hash of the prefix `s[0..i)`, define `h[i+1] = h[i]·B + s[i] (mod M)`. Then the hash of `s[l..r)` is `h[r] − h[l]·B^{r−l} (mod M)`, computed in O(1) after O(n) preprocessing of `h` and the powers of `B`.

Use the raw character code (or `c − 'a' + 1`), never `c − 'a'`: with value 0 for `a`, the strings `a`, `aa`, `aaa` all hash to 0.

2.2 Choosing the modulus

Two equal strings always have equal hashes. Two different strings of length at most `n` collide with probability about `n/M` for a random base. The danger is the *number* of comparisons:

- **Overflow modulo 2^{64}** is broken by Thue–Morse strings for every base: a well-known anti-hash test makes many different substrings collide. Do not use it on judges with adversarial tests.

- **A single prime near 10^9**: inserting 10^5 substring hashes into a set already gives an expected number of colliding pairs around $(10^5)^2/(2\cdot 10^9) = 5$. The birthday paradox makes single small moduli unreliable whenever you compare many hashes against each other.
- **Two primes near 10^9** work but cost twice the time and code.
- **The Mersenne prime $2^{61} - 1$ with a random base** is safe for all realistic sizes and fast: multiply with `__int128` and reduce using $2^{61} \equiv 1$. Pick the base at runtime from a random generator seeded by the clock, so no fixed test can target it.

2.3 Hashing with binary search

If a property is monotone in the length, binary search the length and verify each candidate with hashes.

- **Longest repeated substring**: if a substring of length `L` occurs twice, so does one of length `L − 1`. Check a length with a hash set in O(n); total O(n log n). *AtCoder ABC 141 E Who Says a Pun?* asks for non-overlapping occurrences: store the first position of each hash and check `i − first ≥ L`.
- **LCP of two suffixes**: binary search the largest `L` with equal hashes of length `L`. This yields lexicographic comparison of any two substrings in O(log n): compare the character after the LCP. With that comparator you can sort suffixes, compare rotations, or find the lexicographically smallest of many candidate strings.
- **Palindrome checks**: hash `s` and `reverse(s)`; `s[l..r)` is a palindrome iff its hash equals the hash of the corresponding substring of the reverse, `rev[n−r..n−l)`. *Codeforces 7D Palindrome Degree* rolls both hashes along the prefix to test every prefix in O(1).
- **With point updates**, store hashes in a segment tree whose merge is `H(left)·B^{len(right)} + H(right)`. *CSES 2420 Palindrome Queries* keeps one tree for `s` and one for its reverse.

3. Manacher's algorithm

3.1 The idea

Insert a separator between characters and at both ends (`abba` → `#a#b#b#a#`). Every palindrome of `s` becomes an odd-length palindrome of the new string `t`, centred at a character (odd palindrome of `s`) or at a separator (even palindrome of `s`). Let `d[i]` be the largest radius with `t[i−k..i+k]` a palindrome; then `d[i]` is exactly the length of the corresponding palindrome in `s`, which starts at index `(i − d[i])/2` of `s`.

Maintain the palindrome `[l, r]` with the largest right end found so far. For a new centre `i ≤ r`, its mirror `j = l + r − i` already has `d[j]` computed, and inside `[l, r]` the neighbourhood of `i` looks exactly like the neighbourhood of `j` reversed. So `d[i] ≥ min(d[j], r − i)`. Start from that

value and extend naively. Each successful extension beyond `r` moves `r` to the right, and `r` only increases, so the total work is O(n).

3.2 What it gives you

- The longest palindromic substring (*CSES 1111 Longest Palindrome*).
- The number of palindromic substrings: `Σ ⌈d[i]/2⌉` over all centres of `t`.
- O(1) palindrome tests for any `s[l..r]`: the centre of the substring in `t` must have radius at least its length.
- The longest palindromic prefix (a centre `i` with `i − d[i] = 0`) and suffix.

3.3 Example: Codeforces 1326D2 — Prefix-Suffix Palindrome (hard)

Given `s` (total length up to 10^6), find the longest palindrome `t = a + b` where `a` is a prefix of `s`, `b` is a suffix of `s`, and `|a| + |b| ≤ |s|`.

Observation: let `k` be the largest value with `s[i] = s[n−1−i]` for all `i < k` (and `2k ≤ n`). An optimal answer can always take these `k` characters from both ends. What remains is the middle `m = s[k..n−k)`, and the answer is `s[0..k) + x + s[n−k..n)` where `x` is the longer of the longest palindromic prefix and the longest palindromic suffix of `m`. The longest palindromic prefix of `m` is the last value of the prefix function of `m + '#' + reverse(m)`, or read off Manacher. Linear time overall. This "strip what is forced, then use one tool on the middle" shape is very common at regionals.

4. String DP

4.1 Splitting into pieces

Let `dp[j]` describe the best way to split the prefix `s[0..j)`. A transition chooses the last piece `s[i..j)`. If pieces must be dictionary words, walk a trie from position `i` to enumerate all words starting there (*CSES 1731 Word Combinations*). If pieces must be palindromes, precompute `pal[i][j]` by interval DP (`s[i] = s[j]` and `pal[i+1][j−1]`), then

```
cuts[j] = min over i ≤ j with pal[i][j] of cuts[i−1] + 1   (0 if pal[0][j]).
```

This is $O(n^2)$ time and memory, fine for `n ≤ 5000`. For larger `n`, enumerate only palindromes through Manacher's centres.

4.2 Counting palindromes in ranges

Codeforces 245H Queries for Number of Palindromes: `|s| ≤ 5000` and up to 10^6 queries "how many palindromic substrings in `s[l..r]`?". Precompute `pal[i][j]`, then by inclusion–exclusion on intervals

```
cnt[i][j] = cnt[i+1][j] + cnt[i][j−1] − cnt[i+1][j−1] + pal[i][j].
```

$O(n^2)$ precomputation and O(1) per query.

4.3 DP over the KMP automaton

Count strings of length `n` over an alphabet `Σ` that **do not** contain a pattern `p` of length `m`. While reading a text left to right, the KMP state (the length of the longest prefix of `p` that is a suffix of the text read so far) is all that matters for the future. Build the transition table `go[state][c]` from the prefix function: if `p[state] = c` go to `state + 1`; otherwise follow the failure link, `go[state][c] = go[π[state − 1]][c]` (and 0 from state 0). Because `π[state − 1] < state`, filling rows in increasing order works.

Then `dp[i][state]` counts texts of length `i` ending in that state without having reached state `m`. The number of texts containing `p` is `|Σ|^n − Σ_s dp[n][s]` (*CSES 1112 Required Substring*). Complexity O(n·m·|Σ|). With several patterns, replace the KMP automaton by the Aho–Corasick automaton and mark states that end a pattern. If `n` is huge (10^{18}), the transition is a fixed matrix and matrix exponentiation applies (National track Lecture 7).

5. Reference implementations

5.1 Hashing mod $2^{61} − 1$: longest repeated substring

Prints a longest substring occurring at least twice (occurrences may overlap), or −1.

```cpp
#include <bits/stdc++.h>
using namespace std;
using u64 = unsigned long long;

const u64 MOD = (1ULL << 61) - 1;
u64 mulmod(u64 a, u64 b) {
    __uint128_t c = (__uint128_t)a * b;
    u64 r = (u64)(c & MOD) + (u64)(c >> 61);
    return r >= MOD ? r - MOD : r;
}

struct Hasher {
    vector<u64> h, pw;
    Hasher(const string& s, u64 base) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
```

```
        for (size_t i = 0; i < s.size(); i++) {
            h[i + 1] = (mulmod(h[i], base) + (unsigned char)s[i]) % MOD;
            pw[i + 1] = mulmod(pw[i], base);
        }
    }
    u64 get(int l, int r) const {                 // hash of s[l..r)
        u64 v = h[r] + MOD - mulmod(h[l], pw[r - l]);
        return v >= MOD ? v - MOD : v;
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    u64 base = rng() % (MOD - 1000) + 500;
    Hasher H(s, base);
    auto repeated = [&](int len) -> int {
        unordered_set<u64> seen;
        seen.reserve(2 * n);
        for (int i = 0; i + len <= n; i++)
            if (!seen.insert(H.get(i, i + len)).second) return i;
        return -1;
    };
    int lo = 0, hi = n - 1, bestPos = -1;
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        int p = repeated(mid);
        if (p != -1) lo = mid, bestPos = p; else hi = mid - 1;
    }
    if (lo == 0) cout << -1 << '\n';
    else cout << s.substr(bestPos, lo) << '\n';
}
```

On `banana` it prints `ana`. Verified against brute force on random binary strings.

5.2 Manacher — CSES 1111 Longest Palindrome

```
#include <bits/stdc++.h>
using namespace std;

vector<int> manacher(const string& s) {
    string t = "#";
    for (char c : s) { t += c; t += '#'; }
    int m = t.size();
    vector<int> d(m, 0);
    for (int i = 0, l = 0, r = -1; i < m; i++) {
        int k = (i > r) ? 0 : min(d[l + r - i], r - i);
        while (i - k - 1 >= 0 && i + k + 1 < m && t[i - k - 1] == t[i + k + 1]) k++;
        d[i] = k;
        if (i + k > r) { l = i - k; r = i + k; }
    }
```

```
    return d;
}

int main() {
    string s;
    cin >> s;
    vector<int> d = manacher(s);
    int best = 0, center = 0;
    for (int i = 0; i < (int)d.size(); i++)
        if (d[i] > best) { best = d[i]; center = i; }
    cout << s.substr((center - best) / 2, best) << '\n';
}
```

The CSES sample `aybabtu` gives `bab`. Verified against brute force.

5.3 Minimum palindrome partition

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    int n = s.size();
    vector<vector<char>> pal(n, vector<char>(n, 0));
    for (int len = 1; len <= n; len++)
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            pal[i][j] = s[i] == s[j] && (len <= 2 || pal[i + 1][j - 1]);
        }
    vector<int> cuts(n, INT_MAX);
    for (int j = 0; j < n; j++) {
        if (pal[0][j]) { cuts[j] = 0; continue; }
        for (int i = 1; i <= j; i++)
            if (pal[i][j]) cuts[j] = min(cuts[j], cuts[i - 1] + 1);
    }
    cout << cuts[n - 1] << '\n';
}
```

`aab` needs 1 cut, `abcde` needs 4. Verified against a memoised brute force.

5.4 CSES 1112 — Required Substring (KMP automaton DP)

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1'000'000'007;

int main() {
    int n;
    string p;
```

```
    cin >> n >> p;
    int m = p.size();
    vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int k = pi[i - 1];
        while (k > 0 && p[i] != p[k]) k = pi[k - 1];
        if (p[i] == p[k]) k++;
        pi[i] = k;
    }
    vector<array<int, 26>> go(m);
    for (int s = 0; s < m; s++)
        for (int c = 0; c < 26; c++) {
            if (p[s] - 'A' == c) go[s][c] = s + 1;
            else go[s][c] = (s == 0) ? 0 : go[pi[s - 1]][c];
        }
    vector<long long> dp(m, 0), nd(m);
    dp[0] = 1;
    long long total = 1;
    for (int i = 0; i < n; i++) {
        fill(nd.begin(), nd.end(), 0);
        for (int s = 0; s < m; s++) {
            if (!dp[s]) continue;
            for (int c = 0; c < 26; c++) {
                int t = go[s][c];
                if (t < m) nd[t] = (nd[t] + dp[s]) % MOD;   // t == m: contains p
            }
        }
        swap(dp, nd);
        total = total * 26 % MOD;
    }
    long long avoid = 0;
    for (long long v : dp) avoid = (avoid + v) % MOD;
    cout << (total - avoid + MOD) % MOD << '\n';
}
```

The CSES sample (`n = 6`, `ABCDB`) gives 52; small cases (`n = 4`, `AA` → 1976) match exhaustive enumeration.

6. Common mistakes

- **Unsafe hashing**: overflow mod 2^{64}, a single small prime with many comparisons, or a fixed base on a hackable judge.
- **Character value 0** in the hash.
- **Missing `+ MOD`** in the substring formula.
- **Manacher index mapping**: the palindrome of `s` starts at `(i − d[i])/2`, length `d[i]`.
- **Binary search on a non-monotone property** (e.g. "occurs exactly twice" is not monotone in length).
- **Automaton DP that lets state m continue** and counts texts twice.
- **O(n²) tables for n = 10^5**: 10^{10} bytes. Check memory before choosing interval DP.

- **Forgetting overlapping vs non-overlapping occurrences** in repeated-substring problems.

7. Practice problems

- CSES 1753 — String Matching: https://cses.fi/problemset/task/1753
- CSES 1732 — Finding Borders: https://cses.fi/problemset/task/1732
- CSES 1111 — Longest Palindrome: https://cses.fi/problemset/task/1111
- CSES 2420 — Palindrome Queries: https://cses.fi/problemset/task/2420
- CSES 1112 — Required Substring: https://cses.fi/problemset/task/1112
- CSES 1731 — Word Combinations: https://cses.fi/problemset/task/1731
- AtCoder ABC 141 E — Who Says a Pun?: https://atcoder.jp/contests/abc141/tasks/abc141_e
- Codeforces 1326D2 — Prefix-Suffix Palindrome (Hard version): https://codeforces.com/problemset/problem/1326/D2
- Codeforces 7D — Palindrome Degree: https://codeforces.com/problemset/problem/7/D
- Codeforces 245H — Queries for Number of Palindromes: https://codeforces.com/problemset/problem/245/H
- Codeforces 271D — Good Substrings: https://codeforces.com/problemset/problem/271/D
- Codeforces 1200E — Compress Words: https://codeforces.com/problemset/problem/1200/E
- Kattis `stringmatching`: https://open.kattis.com/problems/stringmatching

8. Further reading

- cp-algorithms: "String Hashing", "Manacher's Algorithm", "Prefix function" (automaton section) (https://cp-algorithms.com).
- Codeforces blog "Anti-hash test" / "On the mathematics behind rolling hashes and anti-hash tests" — why mod 2^{64} fails.
- Dan Gusfield, *Algorithms on Strings, Trees, and Sequences*, Part I.
- National track Lecture 6 (suffix array, suffix automaton) and World Finals track Lecture 8 (eertree, Lyndon factorisation).